

A Practical Performance Comparison of ECC and RSA for Resource-Constrained IoT Devices

Manuel Suárez-Albela *, Tiago M. Fernández-Caramés (Senior Member, IEEE) *, Paula Fraga-Lamas, (Member, IEEE) *, Luis Castedo, (Senior Member, IEEE) *

*Dpt. Computer Engineering, Faculty of Computer Science, Universidade da Coruña, 15071, A Coruña, Spain
Email: m.albela@udc.es, tiago.fernandez@udc.es, paula.fraga@udc.es, luis.castedo@udc.es

Abstract—One of the main concerns regarding Internet of Things (IoT) networks is the security of the communications between IoT nodes and the upper layers of the architecture. Since the vast majority of IoT nodes are constrained in terms of power consumption and computational capabilities, the security of their communications is usually lightly addressed. Thus, new IoT hardware platforms are facing this lack of security by including hardware-accelerated cryptographic capabilities. This brings the opportunity to use advanced security mechanisms such as Transport Layer Security (TLS). In this paper, two of the most used TLS authentication algorithms (i.e., ECDSA and RSA) are compared using a resource-constrained IoT node based on the ESP32 SoC (System-on-Chip). The tests are driven by the concept of security level, allowing for a fair comparison in terms of the computational power required to break each cryptographic algorithm. Specifically, security levels from 80 to 192 were tested. A 512-byte JSON file was downloaded 100 consecutive times while measuring both energy consumption and the average time per transaction. The results show that, although ECDSA outperforms RSA in all the performed tests, real-world scenario testing is needed to determine which security configuration is the best for a hardware platform. Software optimizations and implementations could lead to scenarios where higher security level alternatives outperform simpler and theoretically lighter ones in terms of energy consumption and data throughput. Specifically, the obtained results show that the curve secp256r1 outperforms the curve secp224r1, while providing a higher security level, due to optimizations made on the libraries implementing the ECC operations.

Keywords—IoT security; ECC; ECDSA; RSA; ECDHE; TLS; energy consumption; cryptographic security

I. INTRODUCTION

The rise of the Internet of Things (IoT) paradigm brings the opportunity of having any device connected anytime and everywhere. Such a heterogeneity and ubiquity raise some challenges and threats when compared with other more isolated and controlled communications.

Nowadays, IoT devices are connected to the Internet in diverse environments and sectors [1]–[4]. Nevertheless, security issues pose risks for human safety and privacy [5], [6], and it can be said that the broad adoption of IoT is slowed down because its privacy and security requirements have not been addressed completely [7].

Although IoT devices have many advantages in terms of scalability and cost, they are restricted in terms of memory, battery, computing capabilities and hardware resources, making it impossible to implement the complex and heavy operations needed by ciphering algorithms to encrypt and secure

communications. Furthermore, some resource-constrained IoT devices are deployed in large areas where power outlets are not available [8], and energy harvesting techniques have to be introduced.

Since IoT networks usually rely on TCP/IP, the use of already proven security protocols seems to be the best approach in terms of reliability and implementation efficiency. In this scenario, Transport Layer Security (TLS) arises as the main option, but it has the problem that the most extended cipher suites available were not designed with the limitations of resource-constrained IoT devices in mind [9]. In the last years, UDP-based solutions like DTLS [10] have been introduced in order to provide lightweight communications alternatives. However, when security is added to such solutions, their gains over TLS are diminished, being better to make use of broadly available and optimized TLS implementations [11].

This paper evaluates TLS when using Rivest-Shamir-Adleman (RSA) in terms of security, scalability, power consumption and data throughput. The results are then compared to a more suitable approach for resource-constrained devices using Elliptic Curve Cryptography (ECC) algorithms. The selected ECC key sizes will maintain an acceptable security level for the years to come, unless breakthroughs in ECC cryptography or in certain disruptive technologies (e.g., quantum computing) happen. In fact, the hardware selected for the IoT nodes aims to be as close as possible in terms of computational power to the nodes that will be deployed in the next generation of IoT networks.

The rest of this article is organized as follows. Section II reviews some of the latest academic publications in relation to IoT and security. Moreover, it introduces the basics of TLS and its cipher suites, and analyzes the state of the art related to the performance of RSA and ECC when executed by resource-constrained devices. Section III details the design of the system proposed. In Section IV the experimental setup and the tests performed are described. Finally, Section V is devoted to the conclusions.

II. RELATED WORK

A. Resource-constrained IoT devices and security

In the last years, the number of IoT hardware boards has grown dramatically thanks to the progress made on System-on-Chip (SoC) technologies in terms of energy consumption and

computational power that allowed them to provide computer-like capabilities. Nonetheless, most IoT nodes embed low-power computational devices that can perform specific (but basic) tasks and network communications through the Internet, so they are usually considered as resource-constrained devices [12].

It must be taken into account that IoT devices carry out different tasks depending on their role on the global IoT architecture. To describe such a scenario, Figure 1 depicts a generic IoT architecture where two IoT networks exchange data with services on the Internet conforming a federation between them. Multiple gateways and concentrators exchange data with the IoT nodes in a hierarchical way. Thus, the gateway at the top provides a single point-of-entry to the federated network for the remote users and service providers on the Internet.

It is worth pointing out that the amount of data per transaction increases as ascending from the bottom to the top of the architecture due to data aggregation from the lower layers. Therefore, it can be observed that the simplest IoT devices in terms of hardware would be at the bottom, but more power is required as ascending in the IoT architecture, specially for large deployments where thousands of IoT nodes could exchange data simultaneously.

In such an architecture the weakest link are the IoT nodes, because they have to accommodate the computational demands of robust security schemes, with resource-constrained hardware capabilities. This situation often derives in practice in the fact that IoT developers overlook or address security in a light way, so weak implementations are common, arising public security concerns [7]. For example, in [12] TLS is used and only symmetric key operations are performed, but the cipher suite proposed is currently not recommended since it is considered insecure [13].

In addition, a relevant number of vulnerabilities has been discovered recently after analyzing technologies related to IoT deployments. For instance, in [14] the authors analyze the security of communications protocols used by Wi-Fi, Bluetooth, RFID or ZigBee. Another interesting survey is presented in [15], which addresses the challenges that arise in terms of security in heterogeneous networks.

B. Alternatives for improving IoT security

Several authors have proposed different alternatives to tackle the issues that arise when creating secure mechanisms for IoT networks. Some authors have already provided a good list of the security requirements for IoT technologies [16], while other researchers focused on creating secure IoT architectures [17], [18]. Moreover, some researchers proposed security improvements on the architecture by following a layered approach [19].

Another challenge rises when providing secure end-to-end communications to resource-constrained networks [20]. For such a reason, in the past it was proposed the use of protocols like Datagram TLS (DTLS) [10], which was developed for providing security to UDP-based protocols like Constrained Application Protocol (CoAP) and whose overhead can be

decreased for using it in sensor networks or constrained devices [21]. Note that since CoAP was also designed for resource-constrained devices, its security is addressed lightly, including the use of pre-shared keys or raw public key cipher suites [22]. Certificates can be used in CoAP, but the supported cipher suites have low security levels.

Finally, it is also worth mentioning a growing tendency in IoT security that is based on the use of unique and unclonable physical structures to avoid some attacks [23].

C. TLS for IoT networks and cipher suites

Many IoT systems make use of TCP/IP communications, whose current best securing alternative consists in using TLS [24]. TLS is a standard protocol composed by the TLS Record Protocol and the TLS Handshake Protocol:

- TLS Record Protocol: it provides connection privacy and reliability by using symmetric cryptography (e.g., Advanced Encryption Standard (AES), RC4) and hash functions (e.g., SHA-1).
- TLS Handshake Protocol enables server and client authentication and helps to determine the encryption algorithm and the cryptographic keys used by the application protocol.

Therefore, the TLS Record Protocol is responsible for securing the connection after the TLS Handshake Protocol establishes the parameters of the TLS session. The procedure is conditioned by the cipher suite used, whose name indicates the algorithms involved. For instance, the cipher suite ECDHE-RSA-AES128-GCM-SHA256 makes use of ECDHE-RSA for the key-exchange, AES128-GCM as block cipher and SHA256 as hash algorithm for preserving the integrity of the handshake messages.

RSA and Elliptic curve Diffie-Hellman Ephemeral (ECDHE) are the most popular cipher suites recommended for TLS [13]. The cipher suites based on RSA make use of it for the key-exchange. In contrast, ECDHE-based cipher suites use Ephemeral Diffie-Hellman based on Elliptic Curves for the same purpose.

In order to compare in a fair way ECC and RSA cipher suites, it must be taken into account their security level. Such a level is a value that quantifies the required effort to break a cryptographic primitive [25]. If the effort is 2^k , it is said to offer k -bit security and, therefore, it provides a security level k . For example, a 128-bit security level is achieved by either using 3072-bit RSA or just a 256-bit key size of an ECC curve.

TABLE I: Comparable strengths for symmetric, RSA and ECC ciphers.

Security Level	Symmetric Key Algorithms	RSA Key Size	ECC Curve
80	2TDEA	1024 bits	prime192v1
112	3TDEA	2048 bits	secp224r1
128	AES-128	3072 bits	secp256r1
192	AES-192	7680 bits	secp384r1

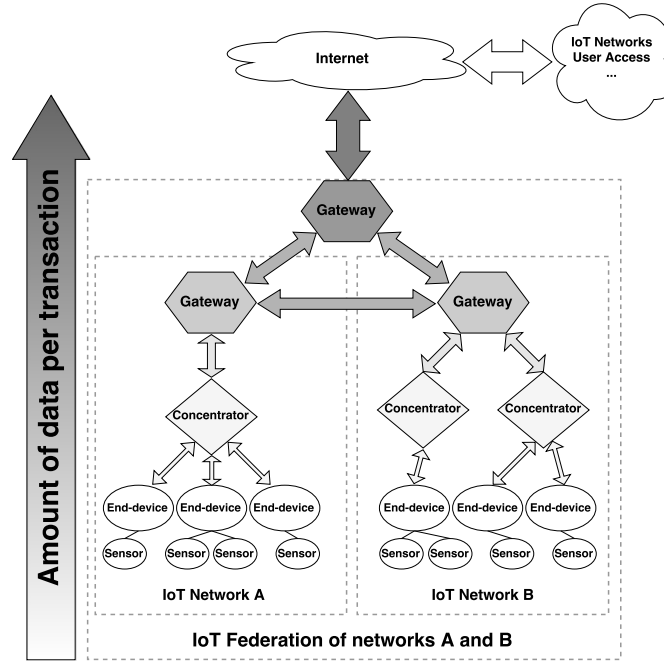


Fig. 1: General IoT architecture.

D. RSA and ECC performance comparisons

Currently, RSA and ECC are not often used by IoT nodes due to their resource demanding requirements, although some researchers proposed resource-efficient hardware implementations of both [26] and evaluated their performance on resource-constrained devices. For example, in [27] the authors evaluated the performance of RSA and ECC on a smart card. Another example is described in [28], where a ECC versus RSA time performance comparison is performed by using 8-bit CPUs. In such a paper the authors conclude that, for key sizes tested, ECC outperforms RSA. It is also worth mentioning a report from ATMEL [29] that compares RSA and ECC for embedded systems and that concludes that, RSA is ten times slower than ECC for a 128-bit security level, while for a 256-bit security level, ECC is between fifty and one hundred times faster.

ECC and RSA have also been studied in terms of power consumption in different scenarios like in [30] and [31], but such studies fall short in some aspects (e.g., security levels may not be taken into account, the energy consumed is just an estimation or the use of outdated hardware platforms and insecure/deprecated cipher suites). It has been found no paper that provides a fair comparison of power consumption in real-world IoT devices with proper security levels, as it is presented in Section IV.

III. SYSTEM OVERVIEW

The proposed testbed architecture is presented in Figure 2. The IoT nodes are implemented by using an ESP32 module [32]. Such a module was chosen mainly due to its resource-constrained hardware with cryptographic support, its wireless interface and its reduced footprint. Moreover, the official firmware supports directly ECDSA and RSA cipher suites, which eases the implementation and testing procedure. In order

to measure the current consumption of the ESP32 during the tests an Orange Pi PC [33] with a INA219 sensor is used [34]. A virtual machine running Debian 9 is utilized to start and coordinate the tests and the energy measurements, to determine the time of each transaction and to persist the obtained results.

To connect all the elements of the testbed, an Asus RT-N12 Wireless N Router was used.

A. Communications architecture

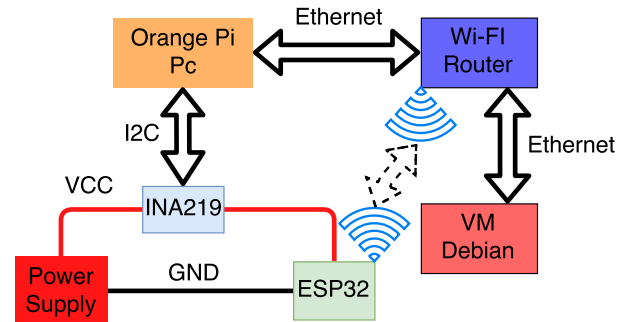


Fig. 2: General testbed architecture.

The presented testbed uses different communications mechanisms to perform the tests and obtain all the required data. The INA219 current sensor is placed between the power supply and the ESP32 to allow for measuring the energy consumed during the tests. An Orange Pi PC accesses the INA219 through the I2C bus. The Orange Pi PC runs an ad-hoc HTTP server that allows for starting, stopping and obtaining the energy consumption results from the Local Area Network (LAN). This allows the virtual machine running Debian to easily coordinate the testing procedure. A Wi-Fi router is used

as central communication node. Both the Orange Pi PC and the virtual machine hosted in a laptop are connected to the router using an Ethernet connection, while the ESP32 is connected using Wi-Fi.

B. Testbed hardware

The ESP32 embeds an IEEE 802.11b/g/n interface and supports Bluetooth v4.2. The core of the SoC is a 32-bit LX6 dual-core microprocessor that operates at up to 240 MHz with 520 KB of SRAM. The hardware acceleration engine for cryptographic algorithms supports AES, SHA-2, RSA, ECC and also presents a Random Number Generator (RNG). It can be powered by either a 3.3 V or a 5.0 V power source or by using a micro-USB connector. The ESP32 module is programmed using ESP32-IDF [35].

The energy measurement subsystem is formed by a INA219 current sensor and an Orange Pi PC to read the values reported by the sensor. The INA219 is capable of measuring voltages of up to 32 VDC and currents of up to 3.2 A. However, it can be configured to measure lower voltage and current ranges. For the tests performed, a maximum value of 800 mA was configured, which allows for a resolution of $18 \mu\text{A}$. The Orange Pi PC is connected to the INA219 through the Inter-Integrated Circuit (I2C) bus. The values are obtained using a Python script that samples the INA219 sensor. Several tests and optimizations to the Python code were performed, obtaining a final sampling rate of 1,000 Hz. The script is launched from a Python HTTP server running in the same Orange Pi PC. The HTTP server provides a REST Application Programming Interface (API), allowing for the current measurement procedure to be managed remotely. Figure 3 shows a picture of the ESP32 together with the current measurement subsystem.

The virtual machine running Debian is the one in charge of starting the tests. A Python script defines the cipher suite to be used, starts the energy measurements using the API running on the Orange Pi PC, and then requests a file from the ESP32 a number of times. The time taken by each of the HTTPS transactions is measured and registered. When all the transactions are finished, it requests the accumulated energy consumption value from the Orange Pi PC and it persists all the data in a JSON file. When the tests end, an HTML page parses the JSON files by using Javascript code and generates charts and tables that present the relevant data of the tests to be further analyzed.

IV. EXPERIMENTS

The aim of the proposed tests is to determine how ECDSA and RSA perform compared to each other and how using different ECC curves and RSA key sizes affect the IoT node communications and energy consumption. For carrying out such tests, OpenSSL was used for both the ESP32, which acts as a server, and the Python script running on the virtual machine that acts as a client. Two different cipher suites were selected according to the NIST guidelines [13]: ECDHE-RSA-AES256-GCM-SHA384 and ECDHE-ECDSA-AES256-GCM-SHA384. A JSON file with a payload of 512 bytes was transferred 100 times using the different certificates generated.

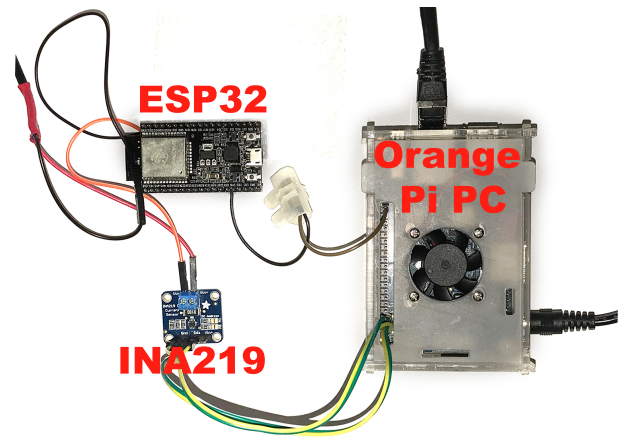


Fig. 3: Energy measurement testbed.

A. Initial setup

For the RSA cipher suite, three certificates with RSA key sizes of 1024, 2048 and 3072 bits were generated, while four certificates with the curves prime192v1, secp224r1, secp256r1 and secp384r1 were generated to be used with the ECDSA cipher suite. These curves and RSA key sizes provide the security levels shown in Table I and they were chosen following the NIST Recommendations for Key Management [36].

Both SHA and AES hardware acceleration were enabled for all the tests, as well as the NIST modulo p optimizations for ECC. It is important to note that the NIST modulo p optimizations affect both cipher suites, since ECC is not only used by the ECDSA algorithm, but also during the ECDHE key exchange phase. For the RSA certificates with key sizes of 1024 and 2048 bits, hardware acceleration for Multi-Precision-Integer (MPI) was used. For the 3072-bit RSA certificate it was not possible to use MPI hardware acceleration. Considering that the ESP32 datasheet [37] states that the crypto engine supports up to 4096-bit operations for RSA, the problem should be software related.

B. Energy consumption

Figure 4 presents the energy consumption values obtained for the different curves and RSA key sizes grouped by their security level. As it can be observed, ECDSA outperforms RSA in all scenarios. For the 128-bit security level scenario, the difference is even greater due to the fact that MPI acceleration must be turned off for the RSA cipher suite to work. Moreover, the obtained results show that for almost the same power consumption (i.e., 21.6 mWh for RSA and 22.4 mWh for ECDSA), ECDSA provides a greater level of security than RSA (192-bit compared with the 112-bit security level of RSA).

Comparing only the energy consumption of the ECDSA cipher suite, it can be seen that the curve secp224r1 performs worse than the curve secp256r1. This is due to the fact that the NIST modulo p optimizations are platform and curve dependent, meaning that even if a curve uses less bits and consequently provides a lower security level, it can be outperformed by a more secure curve. As explained, the actual

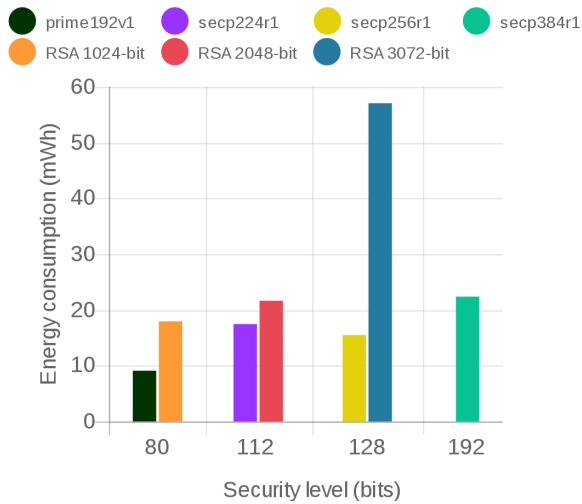


Fig. 4: Energy consumption of the different tested ECC curves and RSA key sizes.

performance of a curve depends on the platform and the platform specific optimizations implemented by the underlying TLS library. To demonstrate that the secp256r1 curve was outperforming the secp224r1 curve due to the platform specific optimizations, the same tests were performed, but this time disabling the NIST modulo p optimizations. The results can be seen in Figure 5. As expected in this case, the less secure secp224r1 curve presents lower power consumption values than the secp256r1 curve.

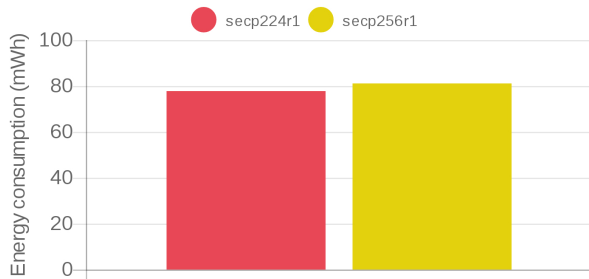


Fig. 5: Energy consumption of secp224r1 and secp256r1 curves without NIST modulo p optimizations.

C. Throughput

Regarding the throughput, the conclusions are very similar to the ones obtained after the energy consumption analysis. Tables II and III present the requests per second achieved during every test performed. ECDSA outperforms RSA in all scenarios. Once again, the secp224r1 curve presents worse results than the secp256r1 curve for the same reason stated in Subsection IV-B. Tables II and III also show the value of multiplying the requests per second by the security level that each ECC curve and RSA key size provides (i.e., $R/s * Sec.Level$). This value represents the trade-off between security and throughput, giving an idea of how efficient is each

TABLE II: Requests per second (R/s) and requests per second times security level ($R/s * Sec.Level$) for ECDSA with different curves.

	prime192v1	secp224r1	secp256r1	secp384r1
R/s	2.28	1.14	1.32	0.92
$R/s * Sec.Level$	182.36	128.04	168.58	177.42

TABLE III: Requests per second (R/s) and requests per second times security level ($R/s * Sec.Level$) for different RSA key sizes.

	RSA 1024-bit	RSA 2048-bit	RSA 3072-bit
R/s	1.17	1.00	0.37
$R/s * Sec.Level$	93.71	111.54	46.94

alternative. After analyzing the tables it can be concluded that all the ECC curves offer a better trade-off between security and throughput than any of the RSA alternatives.

D. Analysis of the results

The results show huge differences for the same security level when comparing ECC and RSA, being ECC a better alternative since it presents better energy consumption and data throughput values. However the results also present interesting findings when different implementations and key sizes of the same algorithm are compared.

Regarding ECC curves, the most interesting finding is that the security level provided by a curve is not always proportional to its performance. With the ESP32-IDF version used for the tests in this article, the secp256r1 curve presented lower energy consumption and higher throughput values than the weakest secp224r1 curve. This fact is explained because the software optimizations performed to speed up the mathematical operations of the algorithms involving the curves are platform and curve dependent.

With respect to the RSA key sizes, the main problem was that for a 3072-bit key size it was not possible to use the hardware acceleration for MPI operations, which it would have a great impact on speeding up RSA calculations and consequently reducing the energy consumption.

Comparing the general performance of the two ciphers suites tested, it can be concluded that for the same security level ECDHE-ECDSA-AES256-GCM-SHA384 always outperforms ECDHE-RSA-AES256-GCM-SHA384 in both energy consumption and throughput.

V. CONCLUSIONS

The aim of this paper was to compare the performance of ECDSA and RSA TLS cipher suites and evaluate the impact of the different ECC curves and RSA key sizes using resource-constrained IoT nodes.

After analyzing all the tests, it can be concluded that, in the selected scenarios, ECDSA presents itself as a better alternative than RSA for securing resource-constrained IoT devices. Moreover, in the near future, when greater security levels will be needed, RSA key sizes will be impractical and ECDSA scalability could guarantee secure IoT deployments for the years to come.

The obtained results also emphasize the importance of testing and measuring the performance of the different algorithms supported by hardware platforms. The created testbed allowed for accurately comparing the different cipher suites alternatives and configurations in terms of security, energy consumption and throughput. For instance, depending on the software implementation and optimizations, weaker alternatives could perform worse than more secure ones. To sum up, real world scenario testing is the only way of accurately finding which security algorithm and configuration is the best fitted for an intended purpose.

ACKNOWLEDGMENT

This work has been funded by the Xunta de Galicia (ED431C 2016-045, ED341D R2016/012, ED431G/01), the Agencia Estatal de Investigación of Spain (TEC2015-69648-REDC, TEC2016-75067-C4-1-R) and ERDF funds of the EU (AEI/FEDER, UE).

REFERENCES

- [1] P. Fraga-Lamas, D. Noceda-Davila, T. M. Fernández-Caramés, M. Díaz-Bouza, and M. Vilar-Montesinos, "Smart Pipe System for a Shipyard 4.0," *Sensors*, vol. 16, no. 12, p. 2186, Dec 2016.
- [2] M. Suárez-Albela, P. Fraga-Lamas, T. M. Fernández-Caramés, A. Dapena, and M. González-López, "Home Automation System Based on Intelligent Transducer Enablers," *Sensors*, vol. 16, no. 10, p. 1595, Sep 2016.
- [3] P. Fraga-Lamas, T. M. Fernández-Caramés, M. Suárez-Albela, L. Castedo, and M. González-López, "A Review on Internet of Things for Defense and Public Safety," *Sensors*, vol. 16, no. 10, p. 1644, Oct 2016.
- [4] O. Blanco-Novoa, T. Fernández-Caramés, P. Fraga-Lamas, and L. Castedo, "An Electricity Price-Aware Open-Source Smart Socket for the Internet of Energy," *Sensors*, vol. 17, no. 3, p. 643, Mar 2017.
- [5] T. M. Fernández-Caramés, P. Fraga-Lamas, M. Suárez-Albela, and L. Castedo, "Reverse engineering and security evaluation of commercial tags for RFID-Based IoT applications," *Sensors*, vol. 17, no. 1, p. 28, Dec 2016.
- [6] P. Fraga-Lamas and T. M. Fernández-Caramés, "Reverse engineering the communications protocol of an RFID public transportation card," in *2017 IEEE International Conference on RFID (RFID)*, May 2017, pp. 30–35.
- [7] A. M. Nia and N. K. Jha, "A comprehensive study of security of Internet-of-Things," *IEEE Transactions on Emerging Topics in Computing*, no. 99, pp. 1–1, 2016.
- [8] J. Pérez-Expósito, T. Fernández-Caramés, P. Fraga-Lamas, and L. Castedo, "VineSens: An Eco-Smart Decision-Support Viticulture System," *Sensors*, vol. 17, no. 3, p. 465, Feb 2017.
- [9] M. Suárez-Albela, T. Fernández-Caramés, P. Fraga-Lamas, and L. Castedo, "A Practical Evaluation of a High-Security Energy-Efficient Gateway for IoT Fog Computing Applications," *Sensors*, vol. 17, no. 9, p. 1978, August 2017.
- [10] Datagram Transport Layer Security Version 1.2. [Online]. Available: <https://tools.ietf.org/html/rfc6347>
- [11] OpenSSL. [Online]. Available: <https://www.openssl.org/>
- [12] A. Sehgal, V. Perelman, S. Kuryla, and J. Schonwalder, "Management of resource constrained devices in the Internet of Things," *IEEE Communications Magazine*, vol. 50, no. 12, pp. 144–149, December 2012.
- [13] T. Polk, K. McKay, and S. Chokhani, "Guidelines for the Selection and Use of Transport Layer Security (TLS) Implementations," *NIST Special Publication 800-52 Revision 1*, no. June, 2005.
- [14] M. Grabovica, S. Popić, D. Pezer, and V. Knežević, "Provided security measures of enabling technologies in Internet of Things (IoT): A survey," in *Proceedings of the 2016 Zooming Innovation in Consumer Electronics International Conference (ZINC)*, Novi Sad, Serbia, 1-2 June 2016, pp. 28–31.
- [15] A. Barki, A. Bouabdallah, S. Gharout, and J. Traoré, "M2M security: Challenges and solutions," *IEEE Communications Surveys Tutorials*, vol. 18, no. 2, pp. 1241–1254, Second quarter 2016.
- [16] E. Vasilomanolakis, J. Daubert, M. Luthra, V. Gazis, A. Wiesmaier, and P. Kikiras, "On the security and privacy of Internet of Things architectures and systems," in *Proceedings of the 2015 International Workshop on Secure Internet of Things (SIoT)*, Vienna, Austria, 21-25 September 2015, pp. 49–57.
- [17] M. Leo, F. Battisti, M. Carli, and A. Neri, "A federated architecture approach for Internet of Things security," in *Proceedings of the 2014 Euro Med Telco Conference (EMTC)*, Naples, Italy, 12-15 November 2014, pp. 1–5.
- [18] I. D. Addo, S. I. Ahamed, S. S. Yau, and A. Buduru, "A reference architecture for improving security and privacy in Internet of Things applications," in *Proceedings of the 2014 IEEE International Conference on Mobile Services*, Alaska, USA, 27 June-2 July 2014, pp. 108–115.
- [19] K. Zhao and L. Ge, "A survey on the Internet of Things security," in *Proceedings of the 2013 Ninth International Conference on Computational Intelligence and Security*, Leshan, China, 14-15 December 2013, pp. 663–667.
- [20] R. Hummen and K. Wehrle, "Standards-based end-to-end IP security for the Internet of Things," in *Proceedings of the 2013 21st IEEE International Conference on Network Protocols (ICNP)*, Göttingen, Germany, 7-10 October 2013, pp. 1–3.
- [21] F. Van den Abeele, V. T. J. Hoebeke, I. Moerman, and P. Demeester, "Secure communication in IP-based wireless sensor networks via a trusted gateway," in *Proceedings of the 2015 IEEE Tenth International Conference on Intelligent Sensors, Sensor Networks and Information Processing (ISSNIP)*, Singapore, 7-9 April 2015, pp. 1–6.
- [22] Using raw public keys in Transport Layer Security (TLS) and Datagram Transport Layer Security (DTLS). [Online]. Available: <https://tools.ietf.org/html/rfc7250>
- [23] U. Rührmair, S. Devadas, and F. Koushanfar, *Security Based on Physical Unclonability and Disorder*. New York, NY: Springer New York, 2012, pp. 65–102.
- [24] The Transport Layer Security (TLS) protocol version 1.2. [Online]. Available: <https://tools.ietf.org/html/rfc5246>
- [25] A. K. Lenstra, T. Kleinjung, and E. Thomé, "Universal security from bits and mips to pools, lakes-and beyond," *Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)*, vol. 8260, no. 1024, pp. 121–124, 2013.
- [26] S. Wu and Y. Zhu, "A resource efficient architecture for RSA and Elliptic Curve Cryptosystems," in *Proceedings of the 2006 International Conference on Communications, Circuits and Systems*, vol. 4, Guilin, Guangxi, China, 25-28 June 2006, pp. 2356–2360.
- [27] M. Savari, M. Montazerolzhour, and Y. E. Thiam, "Comparison of ECC and RSA algorithm in multipurpose smart card application," in *Proceedings of the 2012 International Conference on Cyber Security, Cyber Warfare and Digital Forensic (CyberSec)*, Kuala Lumpur, Malaysia, 26-28 June 2012, pp. 49–53.
- [28] N. Gura, A. Patel, A. Wander, H. Eberle, and S. C. Shantz, "Comparing Elliptic Curve Cryptography and RSA on 8-bit CPUs," pp. 119–132, 2004.
- [29] K. Maletsky, "RSA vs ECC comparison for embedded systems," 2015.
- [30] P. Roberto de Oliveira, V. Delisandra Feltrim, L. Andreia Fondazzi Martimiano, and G. Brasilino Marcal Zononi, "Energy consumption analysis of the cryptographic key generation process of RSA and ECC algorithms in embedded systems," *IEEE Latin America Transactions*, vol. 12, no. 6, pp. 1141–1148, Sept 2014.
- [31] E. Noroozi, J. Kadivar, and S. H. shafiee, "Energy analysis for wireless sensor networks," in *Proceedings of the 2010 2nd International Conference on Mechanical and Electronics Engineering*, vol. 2, Kyoto, Japan, 1-3 August 2010, pp. 382–386.
- [32] ESP32 Overview — Espressif Systems. [Online]. Available: <https://www.espressif.com/en/products/hardware/esp32/overview>
- [33] Orange pi pc - Orangepi. [Online]. Available: <http://www.orangepi.org/orangepipc/>
- [34] INA219 SBOS448G –AUGUST 2008–REVISED DECEMBER 2015. [Online]. Available: <http://www.ti.com/lit/ds/symlink/ina219.pdf>
- [35] Espressif IoT Development Framework. Official development framework for ESP32. [Online]. Available: <https://github.com/espressif/esp-idf>
- [36] E. Barker, W. Barker, W. Burr, W. Polk, and M. Smid, "Recommendation for key management – part 1: General (revision 3)," *NIST Special Publication 800-57 Part 1, Revision 3*, 2012.
- [37] ESP32 Datasheet, Version 2.1. [Online]. Available: https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf